Azure-Zero-Trust-Lab Write-Up

Tahmid Abtahee · Independent Study, COP 4900, University of South Florida · November 7, 2026

Why I Built This

My earlier projects were detection work: malware reverse engineering and packet analysis. Both start after something has already happened. This project goes the other direction and covers access control, which decides who gets in before anything can go wrong.

I scoped it as an identity project. There is no SIEM and logging stays light, so the write-up is mostly tables and text, with a screenshot only where it proves a behavior. The CI/CD piece was added late, after it came up in a technical interview. The flow logs were added at my professor's request.

Lab Environment

The lab ran in one Entra tenant and one Azure subscription. A management group hierarchy sits above the subscription: ZT-Lab at the top with ZT-Dev and ZT-Prod beneath it. The subscription lives under ZT-Dev, and ZT-Prod stayed empty.

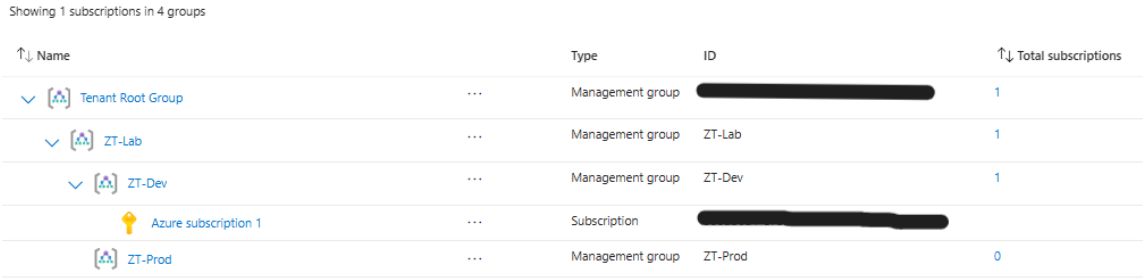


Figure 1: Management groups. Tenant and subscription IDs are blacked out.

Account	Purpose
test-reader	Reader test user
test-contributor	Test user with Contributor, scoped to one subscription
breakglass-admin	Emergency account, excluded from CA001
zt-admin	Administrator account for building the lab

PIM needed Entra ID P2, so I used the P2 free trial. Security Defaults had to be disabled to use custom Conditional Access. Users also needed a usage location set before licenses could be assigned.

The lab was torn down after testing. Every finding below is backed by a screenshot or a table I recorded while it was running.

1. Governance with Azure Policy

Zero Trust mapping: resources are governed centrally, and policy is enforced before a resource exists.

What I Did. I assigned the built-in "Allowed locations" policy at the ZT-Lab management group with a Deny effect and East US as the only allowed region. Everything beneath ZT-Lab inherits it.

What It Showed. Creating a storage account in South Africa North was rejected in the portal with an Allowed locations error. The deny came from the top of the hierarchy, not from a setting on the subscription.



Figure 2: Storage account blocked by the Allowed locations policy.

2. Federation with SAML and OIDC

Zero Trust mapping: every request carries a verifiable identity, and tokens are signed and time-bound.

What I Did. I set up a SAML enterprise app and an OIDC app registration, captured a token from each, and decoded them by hand. For OIDC I used the implicit flow and decoded the ID token at jwt.ms.

What It Showed. Both tokens had a validity window of about 65 minutes. In the OIDC token, `iat` to `exp` is 3900 seconds. The `idp` claim showed an external personal Microsoft account, which is a useful reminder that the token records who vouched for the identity, not just who signed in.

Two limits. Implicit flow is the older pattern, and authorization code with PKCE is the better choice today. The SAML app used placeholder URLs because the test service samltest.id was no longer available.

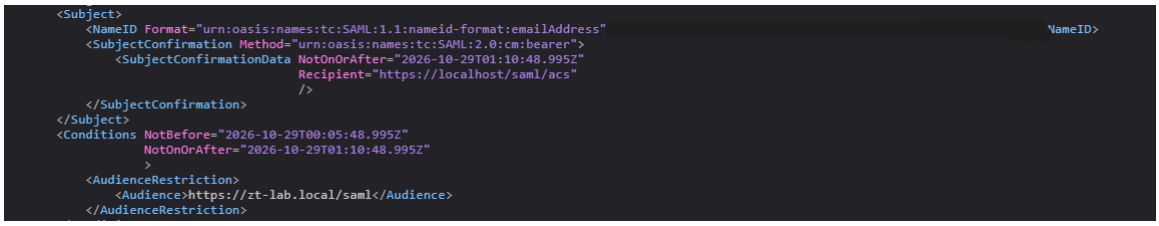


Figure 3: SAML assertion showing the validity window (NotBefore to NotOnOrAfter) and audience. The user identifier is redacted.



Figure 4: Decoded OIDC token claims. IDs are redacted.

3. Conditional Access

Zero Trust mapping: access is decided per request by a policy, not by network location.

What I Did. I created CA001 to block a test user from Azure Resource Manager, with the break-glass account excluded so I could not lock myself out. I ran it in report-only mode first, then switched it to enforced.

What It Showed. In report-only mode the sign-in log recorded Failure with a Block result and the user was not interrupted. Once enforced, the sign-in succeeded but access to the resource was denied with error 53003. A 401 page I hit during testing was an RBAC denial, not Conditional Access, so I did not use it as evidence. The block screen also listed the device state as Unregistered, which nothing in my policy evaluated.

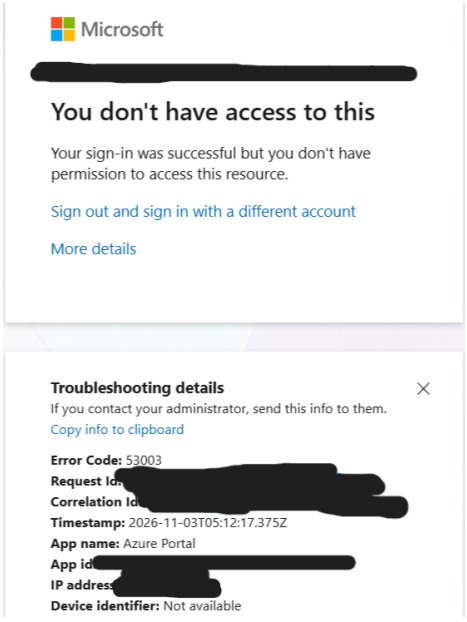


Figure 5: Error 53003, blocked by Conditional Access.

4. Privileged Identity Management

Zero Trust mapping: least privilege, granted per session and removed automatically.

What I Did. I made the test user eligible for Security Reader at the Default Directory scope, with no standing assignment. To test activation I turned CA001 off, then activated the role with a written justification.

What It Showed. The role showed as Activated with an end time, and it expired on its own. The user held no privileged role outside that window.

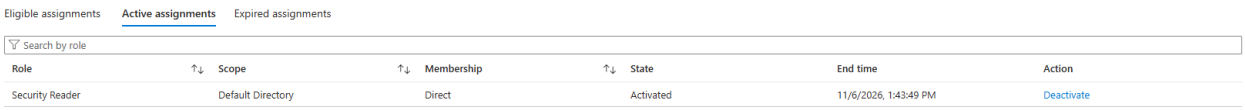


Figure 6: Security Reader activated through PIM, with an automatic end time.

5. Policy as Code

Zero Trust mapping: guardrails are versioned and deployed without long-lived secrets.

What I Did. I wrote the Allowed locations assignment in Bicep at subscription scope. A GitHub Actions workflow deploys it, triggered by workflow_dispatch or a push to `policy-as-code/**`. The workflow signs in with OIDC workload identity federation. I created a federated credential trusted only for the main branch and gave the identity the Resource Policy Contributor role. The only GitHub secrets are three IDs, with no client secret.

What It Showed. The run succeeded and a second run succeeded. The credential is trusted for the main branch only.

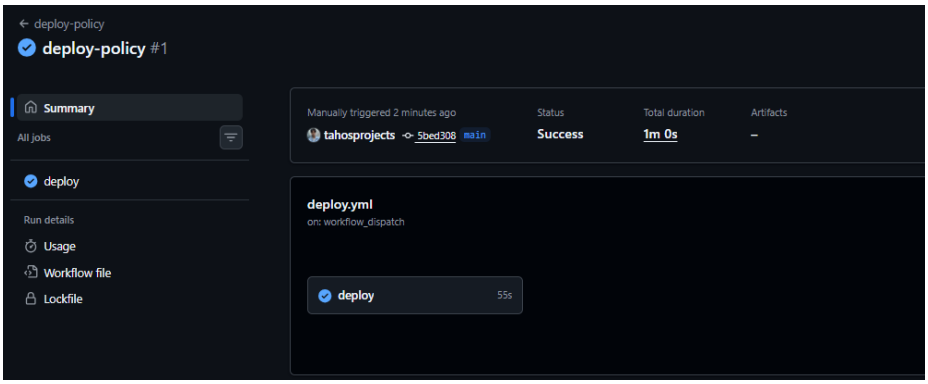


Figure 7: deploy-policy workflow run succeeding.

6. Network Segmentation

Zero Trust mapping: default deny from the internet, with the minimum rule needed for administration. Traffic between the two subnets was still allowed by the default AllowVnetInBound rule, which I did not override.

What I Did. I built a VNet at 10.0.0.0/16 with two subnets, snet-web and snet-app, each with an NSG. I hit a small hiccup creating the /16 address range and worked through it. I then added a deliberately bad rule, Any source to Any destination on any port at priority 100, to see whether anything would flag it.

What It Showed. Defender for Cloud on the free tier gave no recommendation, so detection in this lab came down to manual review of the rule list. I replaced it with HTTPS (443) from a single admin IP at priority 200, and the default DenyAllInBound rule handled the rest. I also enabled a VNet flow log, but it captured nothing because I could not deploy VMs, so there was no traffic to record.



Figure 8: The Any/Any mistake rule at priority 100, above the default AllowVnetInBound rule.

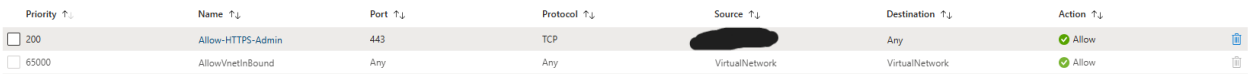


Figure 9: Fixed rule, HTTPS from one admin IP. The source IP is blacked out.

Evidence Summary

Control	Evidence	Type
Governance	Allowed locations deny on storage account	Screenshot
Federation	SAML assertion, decoded OIDC token	Screenshot
Conditional Access (report-only)	Policy state Report-only, result Report-only: Failure, grant control Block	Table only
Conditional Access (enforced)	Error 53003 on sign-in to Azure Resource Manager	Screenshot
PIM	Security Reader activated with end time	Screenshot
Policy as code	Successful deploy-policy run, re-run	Screenshot
Segmentation	NSG rules before and after	Screenshot
Flow logs	Enabled, no traffic captured	Table only
Access review	Created, status Not started	Table only

Zero Trust Assessment

I scored the lab against NIST SP 800-207 and the CISA Zero Trust Maturity Model v2.0. In NIST terms, my reading is that Entra Conditional Access acts as the policy decision point and Azure Resource Manager as the enforcement point. That mapping is my interpretation, not something either document states.

NIST tenet	Rating	Reason
1. All data and services are resources	Partial	Governance covers the subscription, but no workloads were deployed
2. Communication is secured	Partial	Signed tokens. NSGs deny inbound internet traffic by default, subnet-to-subnet traffic left open. Encryption not tested
3. Per-session, least-privilege access	Strong	PIM activation with justification, auto-expiry, time-bound tokens
4. Dynamic policy	Partial	CA001 uses identity and app only, with no device or risk signals
5. Asset posture monitored	Gap	Device state shown as Unregistered and never evaluated
6. Enforced before access	Partial	Enforcement proven, but no MFA was enforced
7. Telemetry improves posture	Gap	Flow log captured no traffic

CISA pillar	Stage	Notes
Identity	Initial	PIM is the strongest function. No MFA, so authentication is Traditional
Devices	Not assessed	Out of scope
Networks	Initial	Segmentation limited to NSG rules, no traffic data
Applications and Workloads	Initial	Policy deployed by CI/CD, no workloads
Data	Not assessed	Out of scope

The biggest single gap is that no MFA was enforced, which means a stolen password would still be enough to sign in. These ratings are a self-assessment based only on what I built and tested. I checked the CISA stage wording against Microsoft's mapping pages rather than CISA's own PDF.

Limitations

Flow logs captured no traffic, so I cannot claim any network visibility. The access review never completed, Defender for Cloud showed no findings on the free tier, and everything ran in a single tenant and subscription. Devices and Data were not assessed.

What I Would Do Next

First, enforce MFA through Conditional Access, then require phishing-resistant strength. Second, add device and sign-in risk conditions so the policy is no longer static. Third, run an access review through a full cycle. Fourth, deploy workloads, generate real traffic, and analyze the flow logs.

After testing I tore the lab down. The repository holds the Bicep file, the workflow, and the screenshots as the record of what was built.